

CYBERSECURITY FOR PROFESSIONALS BOOTCAMP

HANDS-ON LAB

Vulnerability Assessment and Digital Forensics

(All inserted answers have been **Bolded**)

1. Before You Start

This lab runs entirely in your web browser. There is nothing to install, no virtual machine to connect to, no account to create, and no password to request. If you can open a web page and use a spreadsheet, you have everything you need. Read this section fully before Day 1. It takes five minutes and it will save you from every common problem in this lab.

1.1 What you need

Item	Detail
A web browser	Chrome, Edge, Firefox or Safari. Any current version is fine.
A spreadsheet	Excel, Google Sheets or LibreOffice Calc. You will build two tables in it.
A document editor	Word, Google Docs or anything that produces a PDF for your final reports.
This document	All evidence you need is printed inside it. Keep it open beside your browser.

1.2 The three websites you will use

Tool	Address	What you use it for
CyberChef	gchq.github.io/CyberChef	Hashing evidence, decoding a suspicious file, and pulling indicators out of log data. Published by GCHQ. Runs entirely inside your browser - nothing you type is ever uploaded anywhere.
NVD	nvd.nist.gov	The US National Vulnerability Database. You look up which software versions a CVE actually affects. This is how you prove a scanner finding is wrong.
CISA KEV Catalog	cisa.gov/known-exploited-vulnerabilities-catalog	The US government list of vulnerabilities confirmed to be exploited by real attackers. This is the threat intelligence source you re-prioritise against.

If a website will not load

Every piece of evidence you need is printed in this document. The websites confirm what you already have; they do not hold anything you cannot see here.

CyberChef: once the page has loaded once, it keeps working even with your network off. If it will not load at all, Section 9.5 gives you a command built into Windows and macOS that does the same job.

NVD: Appendix A prints the affected-version ranges for every CVE in this lab. Use those and note in your report that you used the printed values.

CISA KEV: Appendix A prints the KEV status and date for every CVE in this lab.

You are never blocked. Move on and keep working.

1.3 How this lab is structured

Session	What you do	What you hand in
Day 1	Triage a real vulnerability scan export, then manually validate the findings that matter.	Severity breakdown, top ten list, five validated findings including at least one false positive.
Day 2	Cross-reference your validated findings against live threat intelligence and re-prioritise, then write the assessment report.	KEV cross-reference table, re-prioritised list, full vulnerability assessment report.
Day 3	One of the hosts you assessed has actually been compromised. Preserve evidence, build a timeline, analyse logs, and find the root cause.	Evidence record, timeline, consolidated IOC list, forensic report.

Days 1 to 2 and Day 3 are one story

The vulnerability you correctly prioritise on Day 2 is the same vulnerability an attacker used on Day 3.

If you rush Days 1 and 2, Day 3 will not make sense. If you do them properly, Day 3 will feel obvious.

2. The Scenario

You are a junior security analyst at Meridian Trust Bank. The bank runs vulnerability assessments on a regular cadence across all environments, not just once before launch.

You have been asked to run this cycle's assessment against a representative slice of the bank's infrastructure, validate what the scanner found, and deliver a report the infrastructure team can actually act on.

2.1 The environment you are assessing

Host	Address	Role	Notes
MTB-WEB01	10.50.2.24	Customer portal web server	DMZ, internet facing, high criticality
MTB-APP01	10.50.2.25	Application and file transfer	Legacy FTP still in use
MTB-FILE01	10.50.2.26	Internal file server	Legacy protocols still enabled
MTB-DC01	10.50.2.10	Windows domain controller	Authenticated scan credentials supplied
MTB-DVWA01	10.50.2.30	Test web application	Non-production, deliberately weak

Scope and ethics

Everything in this lab is evidence that has already been collected for you. You are analysing it, not generating it.

Do not point any scanning tool at any host, website or address anywhere. Not the addresses in this document, not your employer's network, not a site you found online.

The addresses in this lab are internal, private addresses that do not exist on the public internet. The single external address you will see on Day 3 belongs to the scenario, not to you. Do not visit it, scan it, or try to connect to it.

Analysts get fired and prosecuted for scanning things they were not authorised to scan. The habit of checking your authorisation first matters more than any tool in this lab.

3. How to Work Through This Lab

3.1 Time plan

Should take the rest of the afternoon sessions if not, we will use some time in the next day to finish it.

3.2 Set up your workbook before you start

Create one spreadsheet and one document now. You will add to both all week, and having them ready stops you copying things twice.

Where	Name it	What goes in it
Spreadsheet tab 1	All Findings	The 37-row scan export from Section 4.2
Spreadsheet tab 2	Validation	Your five validated findings and verdicts
Spreadsheet tab 3	KEV	The ten CVEs and their KEV status

Where	Name it	What goes in it
Spreadsheet tab 4	Timeline	The Day 3 file system timeline, annotated
Spreadsheet tab 5	IOCs	Your consolidated indicator list
Document 1	MTB Vulnerability Assessment Report	Your Day 2 deliverable
Document 2	MTB Forensic Report - Case MTB-IR-2026-0902	Your Day 3 deliverable

3.3 What a good answer looks like

Most of the marks in this lab are for reasoning, not for reaching the right verdict. Here is the same finding written up three ways.

Standard	Example write-up
Not acceptable	MTB-0016 is a false positive. The version is wrong.
Passing	MTB-0016 (CVE-2017-7494) is a false positive. The CVE affects Samba 3.5.0 up to 4.6.4. MTB-FILE01 runs Samba 4.15.13, which is outside that range.
Strong	MTB-0016 (CVE-2017-7494, SambaCry) is a false positive. NVD states the flaw affects Samba from version 3.5.0 and before 4.6.4, 4.5.10 and 4.4.14. The evidence pack shows MTB-FILE01 running Samba 4.15.13 (smbclient --version, and package samba 2:4.15.13+dfsg-0ubuntu1). 4.15.13 is nine minor versions past the highest affected release, so the host is not vulnerable. The scanner appears to have matched on the presence of a Samba banner rather than on the version. Note this finding IS in the KEV catalog (added 2023-03-30), which is why validating it mattered: unvalidated, it would have been near the top of the remediation queue.

The pattern to copy

Name the finding and the CVE. State the affected range and where you got it. State the detected version and where you got it. State the verdict. Say what the scanner probably did wrong. Note anything that makes the finding interesting beyond the verdict.

Six sentences. Every validated finding in your report should follow that shape.

3.4 Glossary

Terms used in this lab, in the order you will meet them.

Term	What it means
CVE	Common Vulnerabilities and Exposures. A unique identifier for one specific publicly disclosed vulnerability, in the form CVE-2021-41773. The number does not indicate severity.
CVSS	Common Vulnerability Scoring System. A score from 0.0 to 10.0 describing how bad a vulnerability would be if it were exploited. It says nothing about whether anyone is exploiting it.
Severity band	The label attached to a CVSS range: Critical 9.0-10.0, High 7.0-8.9, Medium 4.0-6.9, Low 0.1-3.9, Info 0.0.
Authenticated scan	A scan that logs in to each host with credentials and reads the installed package list, rather than only probing from outside.
False positive	A finding the scanner reported that is not actually present or not actually exploitable on that host.

Term	What it means
True positive	A finding the scanner reported that you have confirmed is genuinely present.
NVD	National Vulnerability Database, run by NIST. The authoritative public record of each CVE, including its description and affected version ranges.
KEV	Known Exploited Vulnerabilities catalog, run by CISA. The authoritative list of vulnerabilities confirmed to be exploited by real attackers in the wild.
Compensating control	A measure that reduces the risk of a vulnerability you cannot fix right now, without fixing the vulnerability itself.
IOC	Indicator of compromise. A concrete, searchable artefact of an intrusion: an IP address, a file path, a file hash, an account name.
Order of volatility	The principle that evidence which disappears fastest must be collected first.
Hash	A fixed-length fingerprint of data. The same input always gives the same hash; any change to the input gives a completely different one.
Webshell	A small script uploaded to a web server that lets an attacker run commands through ordinary web requests.
Beacon	A process on a compromised host that repeatedly connects out to attacker infrastructure, usually on a fixed interval.
Persistence	Anything an attacker installs so their access survives a reboot, a password change, or a process being killed.
Privilege escalation	Moving from a low-privilege account to a higher-privilege one, typically to root or Administrator.
Defanging	Rewriting an address as 185[.]220[.]101[.]47 so it cannot become a clickable link when the IOC list is shared.
Root cause	The specific weakness that allowed the compromise to begin. Not the malware, not the backdoor. The way in.

4. Day 1, Part 1: Triage the Scan

An authenticated vulnerability scan of the five hosts was run on 8 September 2026. It ran for 41 minutes and produced 37 findings. That export is printed in full below.

4.1 What an authenticated scan is, and why it matters

An unauthenticated scan looks at a host from the outside: which ports are open, what banners the services return. An authenticated scan logs in with credentials and reads the actual installed package list.

Authenticated scanning finds far more, because most vulnerabilities are in software that never exposes a banner. It also produces different kinds of false positives, which is exactly what you will be hunting for in Part 2.

4.2 The scan export

Findings are listed in scanner order, grouped by host. The second line of each vulnerability cell is the service and version the scanner detected. You will need those versions in Part 2.

ID	Host	Port	Severity	CVSS	CVE	Vulnerability / Detected version
MTB-0001	MTB-WEB01	80/tcp	Critical	9.8	CVE-2021-42013	Apache HTTP Server Path Traversal and RCE

ID	Host	Port	Severity	CVSS	CVE	Vulnerability / Detected version
						Apache httpd 2.4.49
MTB-0002	MTB-WEB01	80/tcp	High	7.5	CVE-2021-41773	Apache HTTP Server Path Traversal and RCE Apache httpd 2.4.49
MTB-0003	MTB-WEB01	general	High	7.8	CVE-2021-4034	polkit pkexec Local Privilege Escalation (PwnKit) policykit-1 0.105-31
MTB-0004	MTB-WEB01	80/tcp	Medium	6.1	CVE-2020-11022	jQuery Cross Site Scripting jQuery 3.4.1
MTB-0005	MTB-WEB01	80/tcp	Medium	5.3	-	ServerTokens Full Information Disclosure Apache httpd 2.4.49
MTB-0006	MTB-WEB01	80/tcp	Low	3.7	-	Web Server Directory Listing Enabled Apache httpd 2.4.49
MTB-0007	MTB-WEB01	22/tcp	Low	3.7	-	SSH Weak MAC Algorithms Supported OpenSSH 8.9p1
MTB-0008	MTB-WEB01	general	Info	0.0	-	OS Detection: Ubuntu 22.04 LTS Linux kernel 5.15
MTB-0009	MTB-APP01	21/tcp	Critical	9.8	CVE-2011-2523	vsftpd Backdoor Command Execution vsftpd 2.3.4
MTB-0010	MTB-APP01	21/tcp	High	7.5	-	FTP Anonymous Login Permitted vsftpd 2.3.4
MTB-0011	MTB-APP01	21/tcp	High	7.4	-	FTP Credentials Transmitted in Cleartext vsftpd 2.3.4
MTB-0012	MTB-APP01	80/tcp	Critical	9.8	CVE-2014-6271	GNU Bash Shellshock Remote Code Execution GNU bash 5.1.16
MTB-0013	MTB-APP01	22/tcp	Medium	5.3	-	SSH PasswordAuthentication Enabled OpenSSH 8.9p1
MTB-0014	MTB-APP01	general	Medium	6.5	CVE-2023-4911	glibc ld.so Buffer Overflow (Looney Tunables) glibc 2.35
MTB-0015	MTB-APP01	general	Info	0.0	-	OS Detection: Ubuntu 22.04 LTS Linux kernel 5.15
MTB-0016	MTB-FILE01	445/tcp	Critical	9.8	CVE-2017-7494	Samba is_known_pipename RCE (SambaCry) Samba 4.15.13
MTB-0017	MTB-FILE01	23/tcp	High	7.5	-	Telnet Service Detected - Cleartext Protocol telnetd
MTB-0018	MTB-FILE01	445/tcp	High	8.1	-	SMB Signing Not Required Samba 4.15.13
MTB-0019	MTB-FILE01	general	High	7.8	CVE-2021-4034	polkit pkexec Local Privilege Escalation (PwnKit) policykit-1 0.105-31
MTB-0020	MTB-FILE01	445/tcp	Medium	5.0	-	SMB Null Session Enumeration Possible Samba 4.15.13
MTB-0021	MTB-FILE01	22/tcp	Medium	5.3	-	SSH PermitRootLogin Enabled OpenSSH 8.9p1
MTB-0022	MTB-FILE01	general	Low	2.4	-	World-Writable Files Present in /opt Filesystem
MTB-0023	MTB-FILE01	general	Info	0.0	-	OS Detection: Ubuntu 22.04 LTS Linux kernel 5.15
MTB-0024	MTB-DC01	3389/tcp	Critical	9.8	CVE-2019-0708	Remote Desktop Services RCE (BlueKeep) Windows Server 2019

ID	Host	Port	Severity	CVSS	CVE	Vulnerability / Detected version
MTB-0025	MTB-DC01	445/tcp	High	8.8	CVE-2020-1472	Netlogon Elevation of Privilege (Zerologon) <i>Windows Server 2019</i>
MTB-0026	MTB-DC01	3389/tcp	Medium	5.9	-	RDP Weak Encryption Level Configured <i>Windows Server 2019</i>
MTB-0027	MTB-DC01	445/tcp	Medium	5.3	-	SMBv1 Protocol Enabled <i>Windows Server 2019</i>
MTB-0028	MTB-DC01	general	Medium	6.5	-	Local Administrator Password Never Expires <i>Windows Server 2019</i>
MTB-0029	MTB-DC01	general	Low	3.3	-	Defender Real-Time Protection Disabled <i>Windows Server 2019</i>
MTB-0030	MTB-DC01	general	Info	0.0	-	OS Detection: Windows Server 2019 <i>Windows Server 2019</i>
MTB-0031	MTB-DVWA01	80/tcp	Critical	9.8	-	SQL Injection in Application Parameter <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0032	MTB-DVWA01	80/tcp	High	8.8	-	OS Command Injection in Application Parameter <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0033	MTB-DVWA01	80/tcp	High	7.5	-	Unrestricted File Upload Permitted <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0034	MTB-DVWA01	80/tcp	Medium	6.1	-	Reflected Cross Site Scripting <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0035	MTB-DVWA01	80/tcp	Medium	5.3	-	Session Cookie Missing HttpOnly and Secure <i>PHP 8.1</i>
MTB-0036	MTB-DVWA01	80/tcp	Medium	4.3	-	Login Form Served Over Plain HTTP <i>Apache 2.4.52</i>
MTB-0037	MTB-DVWA01	80/tcp	Low	3.1	-	HTTP TRACE Method Enabled <i>Apache 2.4.52</i>

4.3 Step by step

1. Open a new spreadsheet. Create a tab called **All Findings** and copy in the seven columns above. You will reuse this tab all through Days 1 and 2.
2. Count the findings by severity. Fill in the table in Section 4.4. The five counts must add up to 37 - if they do not, you have missed a row.
3. Sort your sheet by CVSS score, highest first. Record the ten highest-severity findings in the table in Section 4.5, along with the host each one affects.
4. Look at your top ten and ask a harder question: does anything here look wrong? You know what these hosts run. Note any finding you suspect the scanner got wrong, and why you suspect it. You are not proving anything yet - you are forming a hypothesis you will test in Part 2.

Why the ranking you just produced is not good enough

You have ranked 37 findings by a number the scanner gave you. Two things are wrong with that ranking, and you will fix them over the next two sessions.

First, some of these findings are not real. Section 5 makes you prove which.

Second, CVSS measures how bad a vulnerability would be if exploited. It says nothing about whether anyone is actually exploiting it. Section 6 fixes that.

4.4 Record: severity breakdown

Severity	Count	Your notes
Critical	6	Highest-severity findings
High	10	Significant findings requiring review
Medium	12	Largest group of findings
Low	5	Lower-priority findings
Info	4	Informational findings
TOTAL	37	Matches the full scan export

4.5 Record: top ten by CVSS

Rank	Finding ID	Host	CVSS	Vulnerability	Suspicious? Why?
1	MTB-0001	MTB-WEB01	9.8	Apache HTTP Server Path Traversal and RCE	No obvious issue
2	MTB-0009	MTB-APP01	9.8	vsftpd Backdoor Command Execution	No obvious issue
3	MTB-0012	MTB-APP01	9.8	GNU Bash Shellshock RCE	Bash 5.1.16 seems much newer than versions associated with Shellshock; verify
4	MTB-0016	MTB-FILE01	9.8	SambaCry RCE	Samba 4.15.13 appears much newer than the vulnerable versions; verify
5	MTB-0024	MTB-DC01	9.8	BlueKeep RCE	host runs Windows Server 2019; verify whether BlueKeep applies to this OS
6	MTB-0031	MTB-DVW A01	9.8	SQL Injection	No obvious issue
7	MTB-0025	MTB-DC01	8.8	Zerologon	No obvious issue
8	MTB-0032	MTB-DVW A01	8.8	OS Command Injection	No obvious issue
9	MTB-0018	MTB-FILE01	8.1	SMB Signing Not Required	No obvious issue

Rank	Finding ID	Host	CVSS	Vulnerability	Suspicious? Why?
10	MTB-0003	MTB-WEB01	7.8	polkit pkexec Local Privilege Escalation (PwnKit)	No obvious issue

5. Day 1, Part 2: Validate the Findings

A scanner does not know things. It matches patterns. When it sees a service banner or a package name, it compares that against a signature and reports whatever the signature says. Signatures are written by people, and people write them loosely.

The result is that a meaningful share of every scan is wrong. Your job as an analyst is to find out which parts, before the infrastructure team spends a weekend patching something that was never vulnerable.

5.1 How analysts validate a finding

You do not need to re-scan anything, and in a real bank you usually cannot. Validation is a comparison:

- **The detected version** - what the scanner says is actually installed on the host. This is in the second line of each vulnerability cell in Section 4.2.
- **The affected version range** - what the vulnerability actually applies to, according to the authoritative source. This is on NVD.

If the detected version falls inside the affected range, the finding is a true positive. If it falls outside, the finding is a false positive and you can say so with evidence.

This is the single most valuable skill in this lab

Anyone can forward a scan report. An analyst is someone who can look at a Critical 9.8 finding and say, with evidence, that it does not apply to this host.

That judgement is what stops a security team from burning its credibility on findings that were never real.

5.2 Looking up a CVE on NVD

1. Go to nvd.nist.gov in your browser.
2. Paste the CVE identifier into the search box at the top of the page and press Enter. You can also go straight to the record by typing the address nvd.nist.gov/vuln/detail/CVE-2021-41773 and swapping in whichever CVE you want.
3. On the CVE page, read the **Description**. For most of the CVEs in this lab the description states the affected versions in plain English, in the last sentence. Read that sentence carefully - it is usually the whole answer.
4. Compare the version named in the description against the detected version from Section 4.2. Write down your conclusion and quote the exact wording you based it on.
5. Optional. Further down the page there is a section called **Known Affected Software Configurations**, which lists the formal version ranges in the form **From (including) 3.5.0 and Up to (excluding) 4.6.4**. Use it to confirm what the description already told you.

Use the description. Do not wait on the version tables.

The Known Affected Software Configurations section is built by a script that runs after the page opens. Until it finishes it displays the words "CPEs loading, please wait." On older CVEs with hundreds of matching product entries it can take a long time, and it sometimes never finishes at all.

That is expected behaviour, not a broken page and not something you did wrong. Do not sit and wait for it.

The plain-English description at the top of the page answers the question in one sentence, and it is the same information. Every CVE in this lab names its affected versions there. Read it, record it, move on.

If you want the formal ranges and the table will not load, Appendix A.1 prints them.

5.3 The validation evidence pack

Your team lead pulled the following detail from the scan logs and from the hosts themselves. This is your supporting evidence. Read it alongside the NVD ranges.

MTB-WEB01 (10.50.2.24)

```
$ curl -sI http://10.50.2.24/
HTTP/1.1 200 OK
Server: Apache/2.4.49 (Ubuntu)

$ dpkg -l | grep -E 'apache2 |policykit'
ii apache2                2.4.49-1ubuntu1      amd64 Apache HTTP Server
ii policykit-1             0.105-31             amd64 authorization framework

$ grep -A2 'ScriptAlias' /etc/apache2/sites-enabled/000-default.conf
ScriptAlias /cgi-bin/ /usr/lib/cgi-bin/
<Directory />
    Require all granted
```

MTB-APP01 (10.50.2.25)

```
$ nc 10.50.2.25 21
220 (vsFTPd 2.3.4)

$ bash --version
GNU bash, version 5.1.16(1)-release (x86_64-pc-linux-gnu)

$ dpkg -l | grep -E 'bash |libc6 '
ii bash                    5.1-6ubuntu1         amd64 GNU Bourne Again SHell
ii libc6                   2.35-0ubuntu3.4      amd64 GNU C Library
```

MTB-FILE01 (10.50.2.26)

```
$ smbclient --version
Version 4.15.13-Ubuntu

$ dpkg -l | grep -E 'samba |policykit'
ii samba                    2:4.15.13+dfsg-0ubuntu1 amd64 SMB/CIFS file server
ii policykit-1             0.105-31             amd64 authorization framework
```

MTB-DC01 (10.50.2.10)

```
PS> Get-ComputerInfo | Select OsName, OsVersion, OsBuildNumber
```

```
OsName       : Microsoft Windows Server 2019 Standard
OsVersion    : 10.0.17763
OsBuildNumber : 17763
```

```
PS> Get-HotFix | Measure-Object
Count : 4      # last cumulative update applied 2020-06-09
```

5.4 Step by step

1. Pick five findings from your top ten. Choose a mix of severities, and make sure you include any finding you flagged as suspicious in Section 4.3.
2. For each one, look up the CVE on NVD using the method in Section 5.2. Findings with no CVE are configuration weaknesses - validate those against the evidence pack in Section 5.3 instead.
3. Write down the affected version range, the detected version, and your verdict. Record all of this in the table in Section 5.6.
4. Identify at least one finding you can prove is a false positive. Explain specifically why the scanner was wrong, quoting the version range and the detected version. "It looks wrong" is not an answer; "the CVE affects Samba 3.5.0 up to 4.6.4 and this host runs 4.15.13" is.
5. Identify at least one finding you can prove is a true positive, with the exact evidence that confirms it.
6. Adjust your severity ranking if validation changed your view of any finding's real risk. Note every change and why you made it.

There is more than one false positive in this scan

The lab asks you to find at least one. There are three.

Two of them are sitting in your top ten, because both were reported as Critical 9.8. That is the point: the findings most likely to be wrong are often the ones screaming loudest for attention.

If you find all three with correct reasoning, say so in your report.

5.5 Where to look first

If you are stuck, these three questions will each lead you to something:

- One host is running a Linux service whose version number is far higher than any version the CVE was ever reported against. Which host, and which service?
- One CVE affects a Windows operating system generation that is older than the one MTB-DC01 is running. Read the NVD description for the RDP finding and compare it against the PowerShell output in Section 5.3.
- One finding names a piece of software that is a full major version ahead of anything the vulnerability applies to. The evidence pack prints its exact version.

5.6 Record: validated findings

Finding ID	CVE	Detected version	Affected range (from NVD)	Verdict	Evidence and reasoning
MTB-0001	CVE-2021-42013	Apache 2.4.49	Apache 2.4.49 and 2.4.50 only	True Positive	NVD states that CVE-2021-42013 affects Apache HTTP Server versions 2.4.49 and 2.4.50 only. MTB-WEB01 is running Apache 2.4.49, which is within the affected range. Therefore, MTB-0001 is a true positive.
MTB-0012	CVE-2014-6271	GNU Bash 5.1.16	GNU Bash through 4.3	False Positive	CVE-2014-6271 affects GNU Bash through version 4.3. MTB-APP01 is running GNU Bash 5.1.16, which is outside the affected range. Therefore, MTB-0012 is a false positive, and the scanner likely matched the Bash package without correctly accounting for the installed version
MTB-0016	CVE-2017-7494	Samba 4.15.13	Samba 3.5.0 up to vulnerable releases before 4.6.4	False Positive	CVE-2017-7494 affects Samba beginning with version 3.5.0 and vulnerable versions prior to the fixed 4.4.14, 4.5.10, and 4.6.4 releases. MTB-FILE01 is running Samba 4.15.13, which is outside the affected range.
MTB-0024	CVE-2019-0708	Windows Server 2019	Windows 7 / Server 2008 / Server 2008 R2	False Positive	CVE-2019-0708 as affecting older Windows systems, including Windows 7, Windows Server 2008, and Windows Server 2008 R2. MTB-DC01 is running Windows Server 2019, which is outside the affected operating system range.
MTB-0025	CVE-2020-1472	Windows Server 2019	Windows Server 2008 R2 through 2019, if unpatched	True Positive	CVE-2020-1472. MTB-DC01 is running Windows Server 2019,

Finding ID	CVE	Detected version	Affected range (from NVD)	Verdict	Evidence and reasoning
					and the evidence pack shows its last cumulative update was June 9, 2020, before the August 2020 security update addressing Zerologon. Therefore, MTB-0025 is a true positive

Give each row several lines in your own document. The reasoning column is where the marks are.

6. Day 2, Part 1: Threat Intelligence Cross-Reference

You now have a list of findings you believe are real. They are still ranked by CVSS, which is a severity score, not a risk score.

CVSS answers: if someone exploited this, how bad would it be? It does not answer: is anyone actually exploiting this? Those are different questions, and a bank has to answer the second one to decide what to fix first.

6.1 What the KEV catalog is

The Cybersecurity and Infrastructure Security Agency maintains the Known Exploited Vulnerabilities catalog: the authoritative list of vulnerabilities confirmed to be exploited in the wild by real attackers.

Under Binding Operational Directive 22-01, US federal civilian agencies are required to remediate KEV-listed vulnerabilities on a fixed deadline. Banks are not bound by that directive, but nearly every large financial institution uses KEV the same way, because it is the single most actionable prioritisation signal available and it is free.

A vulnerability being in KEV means somebody has actually used it against a real organisation. A vulnerability not being in KEV does not mean it is safe - it means there is no confirmed record of exploitation. That distinction matters when you write your report.

6.2 Three ways to check KEV status, in order of reliability

You do not have to browse the catalog. The KEV table on the CISA site is paginated at twenty entries per page across roughly seventy pages, and its filters are not always dependable. Use method one. The other two are there so you always have a way through.

Method 1 - the KEV panel on the NVD page (recommended)

You already have every CVE page open from Section 5. NVD prints the KEV status directly on it, so this costs you no extra page loads.

1. On the CVE page at nvd.nist.gov/vuln/detail/CVE-2017-7494, scroll down past the References table.
2. If the CVE is in KEV, you will see a panel headed **This CVE is in CISA's Known Exploited Vulnerabilities Catalog**, containing four columns: Vulnerability Name, Date Added, Due Date and Required Action.
3. If that panel is not on the page, the CVE is not in KEV. An absent panel is a real result. Record it as such.

4. Record the Date Added and the Due Date.

Method 2 - download the catalog and search it (most reliable)

This is how analysts actually do it, because it works offline and it scales to hundreds of CVEs.

Go to cisa.gov/known-exploited-vulnerabilities-catalog.

Under the paragraph headed **How to use the KEV Catalog**, there is a line reading "The KEV catalog is also available in these formats" followed by links marked **CSV** and **JSON**. Click **CSV**.

Open the downloaded file in Excel, Google Sheets or LibreOffice Calc. It has roughly 1,700 rows, one per vulnerability.

Press Ctrl+F (Cmd+F on a Mac) and search for each CVE identifier. Found means in KEV. Not found means not in KEV.

The columns you need are `cveID`, `dateAdded`, `dueDate` and `knownRansomwareCampaignUse`.

Method 3 - the search box on the CISA page

The catalog page has a search field above the results table. It works, but the page is long and heavily filtered, so if a search returns nothing, confirm with method 1 or 2 before concluding the CVE is absent.

Two date formats, same dates

NVD writes dates American style as MM/DD/YYYY, so 30 March 2023 appears as 03/30/2023.

The CISA catalog and its CSV write dates ISO style as YYYY-MM-DD, so the same day appears as 2023-03-30.

These are the same date. Pick one format for your report and convert everything into it, so your timeline is not ambiguous to whoever reads it.

Check the false positives too

Do not skip the CVEs you disproved in Section 5. Look them up in KEV anyway.

You will find that all three false positives are in the KEV catalog. That is the finding, and it belongs in your report.

A team that had not validated would have put all three at the top of the remediation queue, on the strength of a Critical score plus confirmed in-the-wild exploitation. They would have spent the week patching three things that were never vulnerable, while the finding that actually mattered sat further down the list.

6.3 The ten CVEs in this assessment

CVE	CVSS	In KEV?	Date added	Ransomware ?	Your priority notes
CVE-2021-42013	9.8	Yes	2021-11-03	Known	Critical – true positive, KEV
CVE-2021-41773	7.5	Yes	2021-11-03	Known	High – KEV, internet-facing
CVE-2021-4034	7.8	Yes	2022-06-27	Unknown	High – KEV, privilege escalation

CVE	CVSS	In KEV?	Date added	Ransomware ?	Your priority notes
CVE-2020-11022	6.1	No	—	N/A	Medium – not in KEV
CVE-2011-2523	9.8	No	—	N/A	High – critical RCE, not KEV
CVE-2014-6271	9.8	Yes	2022-01-28	Unknown	False positive – no remediation
CVE-2017-7494	9.8	Yes	2023-03-30	Known	False positive – no remediation
CVE-2023-4911	6.5	Yes	2023-11-21	Unknown	High – KEV, true positive
CVE-2019-0708	9.8	Yes	2021-11-03	Known	False positive – no remediation
CVE-2020-1472	8.8	Yes	2021-11-03	Known	Critical – true positive, KEV, domain controller

6.4 Step by step

1. Check all ten CVEs against KEV and fill in the table above.
2. Flag every validated finding that appears in KEV as actively exploited, regardless of its CVSS score.
3. Re-prioritise your findings list so that confirmed real-world exploitation outweighs CVSS score alone. Remove the false positives from the list entirely - they are not findings.
4. Find at least one case where this reordering moves a finding down. Explain in writing what changed and why. There is a clear one in this data set: a Critical 9.8 finding that is not in KEV, which should now rank below two findings scored lower than it.
5. For your single highest-priority finding, record the CVE publication date and the KEV addition date. Note the gap between them and write two sentences on what that gap tells you about how fast a newly disclosed vulnerability becomes a real-world threat.

6.5 Record: re-prioritised findings

New rank	Finding ID	Host	CVSS	In KEV	Why it sits here
1	MTB-0001	MTB-WEB01	9.8	Yes	KEV, critical RCE, internet-facing web server
2	MTB-0025	MTB-DC01	8.8	Yes	KEV, affects domain controller; very high impact
3	MTB-0002	MTB-WEB01	7.5	Yes	KEV, internet-facing Apache vulnerability
4	MTB-0003	MTB-WEB01	7.8	Yes	KEV, privilege escalation on critical web server
5	MTB-0019	MTB-FILE01	7.8	Yes	KEV, PwnKit privilege escalation
6	MTB-0014	MTB-APP01	6.5	Yes	KEV, glibc version in affected range

New rank	Finding ID	Host	CVSS	In KEV	Why it sits here
7	MTB-0009	MTB-APP01	9.8	No	Critical RCE, but not KEV; moves below exploited findings
8	MTB-0004	MTB-WEB01	6.1	No	Validated XSS, but not KEV and lower impact

CVE: CVE-2021-42013

Publication date: 2021-10-07

KEV date added: 2021-11-03

Gap: 27 days

There was a 27-day gap between publication and addition to the KEV catalog. This shows that a vulnerability can become an actively exploited real-world threat quickly, so critical internet-facing vulnerabilities should be addressed before waiting for KEV confirmation.

7. Day 2, Part 2: Assessment Report and Remediation Plan

Everything so far has been analysis. This section turns it into something the infrastructure team can act on. A report that lists problems without telling anyone what to do about them is not a deliverable.

7.1 What goes in the report

1. List your validated, threat-intelligence-adjusted findings in priority order. Each one needs the affected host, the evidence that confirms it, and a specific recommended fix. "Patch it" is not a fix; "upgrade apache2 to 2.4.51 or later" is.
2. For any finding that cannot be patched immediately, propose a compensating control instead of leaving it open. Examples: network segmentation, disabling the vulnerable feature or module, a tighter firewall rule, removing a service that is no longer needed.
3. Include a section on the false positives. Name each one, state what the scanner claimed, state what you proved, and state the evidence. This section is what tells your reader the rest of the report can be trusted.
4. Recommend a recurring scan cadence for this environment going forward, and justify the frequency you picked. Different hosts may deserve different cadences - MTB-WEB01 is internet facing and MTB-DVWA01 is not.
5. Write a short executive summary, three to five sentences, for a non-technical stakeholder. No CVE numbers, no CVSS scores. What is the risk to the bank, what are you doing about it, and by when.

Executive Summary

The vulnerability assessment identified several security weaknesses across Meridian Trust Bank systems that require remediation. The highest-priority risks involve the internet-facing web server and the domain controller because successful exploitation could lead to unauthorized access, remote code execution, or elevated privileges. Manual validation also identified three scanner findings as false positives, showing the importance of verifying automated scan results before assigning remediation resources. Findings were reprioritized using both technical severity and evidence of real-world exploitation.

7.2 Compensating controls: worked example

Suppose MTB-FILE01 cannot have Telnet removed this month because a legacy backup job still uses it. You cannot fix the finding, so you reduce what an attacker could do with it:

- Restrict the Telnet port to the single management subnet with a firewall rule, so it is not reachable from user or guest zones.
- Alert on any successful Telnet authentication from outside that subnet, so an attempt is visible even if it succeeds.
- Record a dated commitment to migrate the backup job to SSH, with a named owner, so the control does not quietly become permanent.

That is a compensating control: the vulnerability is still there, the exposure is smaller, the risk is being tracked rather than forgotten.

Write the executive summary last, and write it for a person

The stakeholder reading it does not know what CVSS is and does not need to. They need to know whether the bank is exposed, how badly, and what happens next.

A good test: if you read your summary aloud to someone outside IT and they can repeat back what the risk is and what you are doing about it, it works.

Compensating Controls

MTB-WEB01 – Apache vulnerability:

If Apache cannot be upgraded immediately, restrict unnecessary access to the web server, disable unused CGI functionality, and use firewall/WAF rules to block suspicious path-traversal requests until the server can be patched.

MTB-APP01 – Legacy FTP service:

If FTP cannot be removed immediately because a legacy application still depends on it, restrict port 21 to only approved internal systems, disable anonymous access, and monitor FTP activity for suspicious connections.

MTB-DC01 – Zerologon:

If the domain controller cannot be patched immediately, restrict Netlogon traffic to trusted internal systems and isolate unnecessary network access to the domain controller until the required security update can be installed.

7.3 Report structure to follow

Section	Contents
Executive summary	The vulnerability assessment identified several security weaknesses across Meridian Trust Bank systems that require remediation. The highest-priority risks involve the internet-facing web server and the domain controller because successful exploitation could lead to unauthorized access, remote code execution, or elevated privileges. Manual validation also identified three scanner findings as false positives, showing the importance of verifying automated scan results before assigning remediation resources. Findings were reprioritized using both technical severity and evidence of real-world exploitation.
Scope and method	An authenticated vulnerability scan was conducted on September 8, 2026 against five Meridian Trust Bank systems: MTB-WEB01, MTB-APP01, MTB-FILE01, MTB-DC01, and MTB-DVWA01. The scan ran for 41 minutes and produced 37 findings. Selected findings were manually validated by comparing detected software versions against affected version ranges published in NVD and against the host evidence provided in the lab. The CVE findings were then cross-referenced

Section	Contents
	with the CISA Known Exploited Vulnerabilities catalog to account for confirmed real-world exploitation and reprioritize remediation efforts.
Prioritised findings	1. MTB-0001 – Apache HTTP Server Path Traversal and RCE – MTB-WEB01 MTB-WEB01 is running Apache 2.4.49, which falls within the affected range for CVE-2021-42013. The host is also internet-facing, and the vulnerability is listed in CISA KEV. Upgrade Apache from 2.4.49 to 2.4.51 or later. The Apache configuration should also be reviewed so unnecessary CGI functionality is disabled and access outside intended directories is restricted. This finding is ranked first because it combines confirmed vulnerability, internet exposure, critical impact, and known exploitation. 2. MTB-0025 – Zerologon – MTB-DC01 MTB-DC01 is running Windows Server 2019, which is within the affected operating system range for CVE-2020-1472. The evidence shows its last cumulative update was June 9, 2020, before the August 2020 security fix. Install the applicable Microsoft security updates and bring the domain controller to a current supported cumulative update level. This is ranked second because it affects the domain controller and is listed in KEV, creating a risk of domain-level privilege escalation. 3. MTB-0002 – Apache Path Traversal – MTB-WEB01 MTB-WEB01 is running Apache 2.4.49, the affected version for CVE-2021-41773. Upgrade Apache to 2.4.51 or later and review the server's directory-access configuration. This finding is ranked highly because the server is internet-facing and the vulnerability is listed in KEV. 4. MTB-0003 – PwnKit Privilege Escalation – MTB-WEB01 The evidence shows policykit-1 version 0.105-31 installed on MTB-WEB01. The lab's NVD reference identifies this version as within the affected range for CVE-2021-4034. Upgrade the policykit/polkit package to a vendor-supported version containing the January 2022 security fix. This finding is high priority because it is in KEV and could allow an attacker with initial access to escalate privileges. 5. MTB-0019 – PwnKit Privilege Escalation – MTB-FILE01 MTB-FILE01 also contains policykit-1 version 0.105-31, placing it within the affected range for CVE-2021-4034. Upgrade the policykit/polkit package to a vendor-supported release containing the January 2022 fix. It ranks below the WEB01 instance because MTB-FILE01 is an internal server rather than an internet-facing system. 6. MTB-0014 – Looney Tunables – MTB-APP01 MTB-APP01 is running glibc 2.35, which the lab reference identifies as within the affected range for CVE-2023-4911. Upgrade the libc6/glibc package from the installed version to the current vendor-patched Ubuntu release that addresses CVE-2023-4911. This finding remains high priority because it is listed in KEV and affects a production application server. 7. MTB-0009 – vsftpd Backdoor Command Execution – MTB-APP01 MTB-APP01 is running vsftpd 2.3.4, the specific release identified as affected by

Section	Contents
	CVE-2011-2523. Remove version 2.3.4 and replace it with a current supported FTP package; where possible, migrate file transfers to SFTP/SSH and remove the legacy FTP service. Although this finding has a critical severity, it ranks below the KEV findings because it is not currently listed in KEV. 8. MTB-0004 – jQuery Cross-Site Scripting – MTB-WEB01 MTB-WEB01 uses jQuery 3.4.1, while the affected range for CVE-2020-11022 includes versions before 3.5.0. Upgrade jQuery to 3.5.0 or later and test the customer portal for compatibility after the upgrade. This finding ranks lower because it has lower technical impact and is not listed in KEV.
False positives	MTB-0012 – CVE-2014-6271 Shellshock: The scanner reported GNU Bash Shellshock on MTB-APP01. NVD states that the vulnerability affects GNU Bash through version 4.3, while the evidence shows MTB-APP01 running Bash 5.1.16. The installed version is outside the affected range, so the finding is a false positive. MTB-0016 – CVE-2017-7494 SambaCry: The scanner reported SambaCry on MTB-FILE01. NVD identifies vulnerable Samba releases beginning with 3.5.0 and prior to the applicable fixed 4.4.14, 4.5.10, and 4.6.4 releases. MTB-FILE01 is running Samba 4.15.13, which is outside the affected range, so the finding is a false positive. MTB-0024 – CVE-2019-0708 BlueKeep: The scanner reported BlueKeep on MTB-DC01. NVD lists older Windows systems including Windows 7, Windows Server 2008, and Windows Server 2008 R2 as affected. MTB-DC01 runs Windows Server 2019, so the finding does not apply and is a false positive. All three CVEs are listed in CISA KEV, which demonstrates why manual validation was important. Without validation, these findings could have consumed remediation resources even though the affected software or operating system was not present on the hosts.
Compensating controls	MTB-WEB01: If Apache cannot be upgraded immediately, restrict unnecessary access, disable unused CGI functionality, and use firewall/WAF rules to block suspicious path-traversal requests. MTB-APP01: If legacy FTP must remain temporarily, restrict port 21 to approved internal systems, disable anonymous access, and monitor FTP activity. MTB-DC01: If Zerologon cannot be patched immediately, restrict Netlogon traffic to trusted internal systems and limit unnecessary network access to the domain controller until the required security update is installed.
Recommended cadence	MTB-WEB01 should be scanned weekly because it is internet-facing, hosts the customer portal, and has a higher likelihood of exposure to external threats. MTB-DC01 should also be scanned weekly because compromise of the domain controller could have organization-wide impact. MTB-APP01 and MTB-FILE01

Section	Contents
	should be scanned monthly because they are internal systems but still contain legacy services and software that require continued monitoring. MTB-DVWA01 can be scanned quarterly or after significant configuration changes because it is a non-production test system that is deliberately vulnerable. All systems should also be rescanned after major patches or remediation to confirm that identified vulnerabilities have been successfully addressed.

8. Day 3, Part 1: The Alert and Volatile Evidence

Six days after your assessment, one of the hosts you looked at triggered a security monitoring alert. Your team lead believes it has actually been compromised, most likely through one of the vulnerabilities you flagged.

You have been assigned to investigate: preserve the evidence properly, determine exactly what happened, and produce a report that would hold up if a regulator asked to see it.

8.1 The alert

```

MERIDIAN TRUST BANK - SECURITY OPERATIONS CENTRE
ALERT ID      : SOC-2026-09-02-0416
RAISED       : 2026-09-02 09:47:12 UTC
SEVERITY      : High
ASSET        : MTB-WEB01 (10.50.2.24) - DMZ customer portal
RULE         : Periodic outbound connection to non-allowlisted destination

DETAIL        : Host has established an outbound TCP session to
                185.220.101.47 on port 8443 at a consistent 5 minute
                interval since approximately 02:31 UTC. Destination is
                not present in any approved egress allowlist.

                A root-owned process named 'apache-worker' is running.
                This process name does not appear in the Meridian Trust
                standard Apache build manifest.

ACTION        : Host isolated by network ACL at 10:05 UTC.
                HOST LEFT POWERED ON at the request of the IR team.

```

8.2 Order of volatility

Evidence disappears at different speeds. Some of it is gone the moment you pull the power; some of it will still be there in a year. The order in which you collect evidence therefore matters enormously, and getting it wrong destroys information you can never get back.

Order	Evidence type	Lifetime	Examples
1	CPU registers and cache	Nanoseconds	Not practically collectable
2	Memory contents	Until power off	Running processes, open sockets, decrypted data, injected code
3	Network state	Seconds to minutes	Active connections, listening ports, ARP and routing tables
4	Running system state	Until reboot	Logged-in users, open files, loaded modules, running services

Order	Evidence type	Lifetime	Examples
5	Disk contents	Until overwritten	Files, logs, deleted data still in unallocated space
6	Remote and archived logs	Weeks to years	SIEM records, backups, off-host log copies

This is why the host was left powered on

The IR team's instruction to leave MTB-WEB01 running was deliberate. Pulling the power would have destroyed everything in categories 2, 3 and 4 instantly and permanently.

The running process, the established outbound session, and the open file handles that tie them together only exist while the machine is on. Imaging the disk first and then powering off would have produced a technically perfect disk image of an incident you could no longer fully explain.

Note that collecting volatile evidence does change the system slightly - your own commands create processes and write entries. That is accepted practice. You document what you ran and when, so the change is on the record rather than hidden.

8.3 The volatile capture

The on-call responder captured the following from MTB-WEB01 while it was still running, before anything was imaged. Each capture is timestamped. Read them in the order they were taken.

Capture 1 - running processes, 10:12:41 UTC

```
$ ps -ef
UID      PID  PPID  C  STIME TTY      TIME    CMD
root      1      0  0 Sep01 ?        00:00:04 /sbin/init
root     812      1  0 Sep01 ?        00:00:00 /usr/sbin/cron -f
root     844      1  0 Sep01 ?        00:00:01 /usr/sbin/sshd -D
root     901      1  0 Sep01 ?        00:00:02 /usr/sbin/apache2 -k start
www-data 903     901  0 Sep01 ?        00:00:00 /usr/sbin/apache2 -k start
www-data 904     901  0 Sep01 ?        00:00:00 /usr/sbin/apache2 -k start
root    1244      1  0 02:30 ?        00:00:00 /bin/bash /usr/local/bin/apache-worker
root    1247    1244  0 02:30 ?        00:00:00 sleep 300
mtbadmin 1401     844  0 09:47 ?        00:00:00 sshd: mtbadmin@pts/0
```

Capture 2 - network connections, 10:13:02 UTC

```
$ ss -tunap
Netid State  Local Address:Port    Peer Address:Port    Process
tcp    LISTEN  0.0.0.0:22           0.0.0.0:*             users:(("sshd",pid=844))
tcp    LISTEN  0.0.0.0:80           0.0.0.0:*             users:(("apache2",pid=901))
tcp    ESTAB   10.50.2.24:49882     185.220.101.47:8443   users:(("apache-worker",pid=1244))
tcp    ESTAB   10.50.2.24:22        10.50.1.31:50110      users:(("sshd",pid=1401))
```

Capture 3 - logged-in users and login history, 10:13:20 UTC

```
$ who
mtbadmin pts/0          2026-09-02 09:47 (10.50.1.31)
```

```
$ last -n 8
mtbadmin pts/0      10.50.1.31      Wed Sep  2 09:47    still logged in
root      pts/1      185.220.101.47  Wed Sep  2 02:24 - 02:44 (00:20)
mtbadmin pts/0      10.50.1.31      Tue Sep  1 09:14 - 11:52 (02:38)
reboot    system boot 5.15.0-91      Tue May 19 03:08    still running
```

Capture 4 - open network files, 10:13:44 UTC

```
$ lsof -i -n -P
COMMAND      PID      USER   FD  TYPE  NODE  NAME
sshd          844      root    3u   IPv4  TCP   *:22 (LISTEN)
apache2       901      root    4u   IPv4  TCP   *:80 (LISTEN)
apache-worker 1244     root    3u   IPv4  TCP   10.50.2.24:49882->185.220.101.47:8443 (ESTABLISHED)
sshd          1401     root    3u   IPv4  TCP   10.50.2.24:22->10.50.1.31:50110 (ESTABLISHED)
```

Capture 5 - scheduled tasks, 10:14:05 UTC

```
$ cat /etc/cron.d/*
# Apache log maintenance
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
*/5 * * * * root /usr/local/bin/apache-worker >/dev/null 2>&1
```

8.4 Pulling indicators out with CyberChef

You could read through the captures by hand and copy addresses into a list. On a real incident with megabytes of logs you cannot. CyberChef does it in one step, and you will use the same technique on the much larger log files in Section 11.

1. Open gchq.github.io/CyberChef in your browser. The screen has four panels: **Operations** on the left, **Recipe** next to it, **Input** top right, **Output** bottom right.
2. In the Operations search box at the top left, type **Extract IP**. The operation **Extract IP addresses** appears under the **Extractors** category.
3. Drag **Extract IP addresses** from Operations into the Recipe panel. You can also double-click it.
4. Copy the text of Captures 1 to 5 above and paste it into the **Input** panel.
5. The Output panel updates immediately with every IP address found. Tick the **Unique** checkbox in the operation's settings to collapse duplicates.
6. Add a second operation: search for **Sort**, drag **Sort** below Extract IP addresses in the recipe. Your output is now a clean, deduplicated, sorted address list.

Nothing you paste into CyberChef leaves your computer

CyberChef runs entirely as JavaScript inside your own browser. The project documentation states plainly that no recipe configuration and no input, whether text or files, is ever sent to the CyberChef web server.

That property is why it is one of the few public web tools a bank will let an analyst paste evidence into. Every other online decoder and hash calculator uploads what you give it. Know the difference, and check before you paste anything real into a website.

8.5 Step by step

1. Record exactly what triggered the alert: the process, the connection, and the behaviour that was flagged. Quote the alert.

MTB-WEB01 was making outbound TCP connections to 185.220.101.47:8443 every five minutes. A root-owned process named apache-worker, which was not part of the approved Apache build, was responsible for the connections.

2. Working from the five captures, write down the host's state at the time of capture: what was running, what it was connected to, and who was logged in.

The host had a suspicious root-owned apache-worker process running from /usr/local/bin/apache-worker. It maintained a connection to 185.220.101.47:8443, and a cron job was configured to execute the process every five minutes.

3. List every indicator of compromise visible so far - process names, IP addresses, ports, file paths, account names. Use CyberChef for the addresses.

apache-worker, 185.220.101.47, port 8443, /usr/local/bin/apache-worker, root account, /etc/cron.d/apache-logrotate

4. Find the one login in Capture 3 that should not be there, and say what makes it stand out. Compare its source address against everything else in the captures.

A root login from 185.220.101.47 between approximately 02:24 and 02:44 UTC is suspicious because it came from an external IP address. Normal administrator activity came from the internal address 10.50.1.31.

5. Document your evidence-handling steps in the order they were performed, and explain why volatile data was captured before anything else. Reference the volatility table in Section 8.2.

Running processes, network connections, logged-in users, and other volatile data may disappear if the host is rebooted or powered off. Collecting this evidence first preserves information that could otherwise be permanently lost.

8.6 Record: initial indicators

Type	Value	Where you saw it	Why it is suspicious
Process	apache-worker	Capture 1 / ps -ef	Root-owned process not part of the approved Apache build
IP address	185.220.101.47	Captures 2–4 / network and login data	External IP tied to outbound beaconing and suspicious root login
Port	8443	Capture 2 / ss and Capture 4 / lsof	Used by the suspicious outbound connection
File path	/usr/local/bin/apache-worker	Capture 1 and Capture 5	Unauthorized executable associated with the suspicious process

Type	Value	Where you saw it	Why it is suspicious
Account	root	Capture 3 / login history	Root logged in remotely from the suspicious external IP
Scheduled task	/etc/cron.d/apache-logrotate	Capture 5 / cron data	Executes apache-worker every five minutes, indicating persistence

9. Day 3, Part 2: Evidence Integrity

The disk of MTB-WEB01 was imaged at 10:31 UTC, after the volatile captures were complete. In a physical lab you would run an imaging tool and produce a file. What matters far more than the tool is what you do immediately afterwards: prove the image has not changed since you took it.

9.1 Why hashing is the whole ballgame

A cryptographic hash turns any amount of data into a fixed-length fingerprint. Two properties make it useful as evidence:

- The same input always produces the same hash. Anyone can repeat your calculation and get your answer.
- Changing the input by even one character produces a completely different hash, with no resemblance to the original.

Together those mean a hash recorded at acquisition proves the evidence was not altered afterwards. Without it, a defence lawyer, a regulator or an internal auditor can ask whether you modified the evidence, and you have no answer.

A correct conclusion built on unverified evidence is worth nothing

This is the part of forensics that new analysts skip because it feels like paperwork. It is not paperwork. It is the difference between a finding that stands up and a finding that gets thrown out.

Your grade on this lab reflects that. Your evidence handling and hash verification carry as much weight as your conclusions.

9.2 The acquisition record

```
CHAIN OF CUSTODY - CASE MTB-IR-2026-0902

Case           : MTB-IR-2026-0902
Affected asset  : MTB-WEB01 (10.50.2.24), DMZ customer portal
Alert received  : 2026-09-02 09:47 UTC
Host isolated   : 2026-09-02 10:05 UTC (network ACL, host left powered on)
Volatile capture: 2026-09-02 10:12 - 10:14 UTC
Disk acquired   : 2026-09-02 10:31 UTC
Acquired by     : Meridian Trust Bank IR Team
Media           : /dev/sda, 1 GB, MBR, single ext4 partition
Released to     : Bootcamp forensic analyst (you)

ARTEFACT HASHES RECORDED AT ACQUISITION (SHA-256)

/var/www/html/uploads/style.php
2dfc22e2fb93187b7537d9fffec52ecc94fd6416c84b78e68ff7a025e975c20c

/usr/local/bin/apache-worker
1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b

/etc/cron.d/apache-logrotate
d9bc0ee387921dcaedfc87de4a72ec42e45cf268adb2777d5b014375417cb9cc

/etc/passwd
0243860670744531f70b46cfb844a66eb8ca0de0e4b7b15dff991b3960577295
```

Those four hashes are part of your evidence. They go in your IOC list in Section 11, because a hash is how another team detects the same artefact on a different machine.

9.3 Verify a hash yourself in CyberChef

You will now perform the acquisition-and-verification cycle on a small evidence block, so you have done it rather than only read about it.

1. Open gchq.github.io/CyberChef. Clear the Recipe panel if anything is left from Section 8.
2. Search Operations for **SHA2** and drag **SHA2** into the Recipe. It is in the **Hashing** category.
3. In the SHA2 operation, make sure **Size** is set to **256**. That is the default.
4. Type this exactly into the Input panel, on one line, with no space or line break before or after it:

```
MTB-IR-2026-0902-EVIDENCE-BLOCK-01
```

5. Compare your Output against the recorded value below. They must match character for character.

```
62819c23c68b1129f103ed156fbd8561d885d8e3a1246f439b941e73675061f
```

MATCHED

If your hash does not match

You have a stray space, a line break, or an autocorrected character. This is the single most common problem in this section and it is easy to fix.

Click into the Input panel, press Ctrl+A then Delete to clear it completely, and type the line by hand rather than pasting it. Copying from a PDF often brings invisible characters with it.

Check the character count shown at the bottom of the Input panel. It should read 34.

This is not a distraction from the lab. Evidence integrity failing because of an invisible character is a real thing that happens to real analysts, and recognising it is part of the skill.

9.4 Prove that tampering is detectable

1. Change the Input to end in **02** instead of **01**, so it reads **MTB-IR-2026-0902-EVIDENCE-BLOCK-02**.
2. Look at the new Output. Confirm it matches the value below.

```
164cb44525cb26d808d571c8648410d534777b92a136ef2ae2892f26c02c117c
```

3. Compare the two hashes side by side. You changed one character out of 34. Write down how many characters of the hash stayed the same.

3 characters

4. Write two sentences explaining what this demonstrates about why a hash recorded at acquisition is sufficient to prove evidence has not been altered.

Changing only one character in the evidence produced a completely different SHA-256 hash. This demonstrates that a hash recorded at acquisition can be compared later to prove whether evidence has remained unchanged or has been altered.

9.5 Doing the same thing without a browser

Every analyst should be able to hash a file without any tool beyond the operating system. These commands are built in and require nothing to be installed.

Platform	Command
Windows (Command Prompt)	certutil -hashfile evidence.txt SHA256
Windows (PowerShell)	Get-FileHash evidence.txt -Algorithm SHA256
macOS (Terminal)	shasum -a 256 evidence.txt
Linux (Terminal)	sha256sum evidence.txt

If CyberChef is unavailable for any reason, save the evidence block from Section 9.3 into a plain text file with no trailing newline and use one of these commands instead. Note in your report which method you used.

```
PS C:\WINDOWS\system32> cd $HOME
PS C:\Users\kalin> [System.IO.File]::WriteAllText("$HOME\evidence.txt", "MTB-IR-2026-0902-EVIDENCE-BLOCK-01", [System.Text.Encoding]::ASCII)
PS C:\Users\kalin> Get-FileHash "$HOME\evidence.txt" -Algorithm SHA256
```

Algorithm	Hash	Path
SHA256	62819C23C68B1129F103ED156FBDC8561D885D8E3A1246F439B941E73675061F	C:\Users\kalin\evidence.txt

10. Day 3, Part 3: File System Timeline

Once the image was verified, the file system metadata was extracted into a timeline. Every file on a Linux file system carries four timestamps, and reading them in order reconstructs what happened on the host without needing anyone to have written a log.

10.1 Reading a forensic timeline

Letter	Timestamp	What updates it
m	Modified	The file's contents were changed
a	Accessed	The file was read
c	Changed	The file's metadata changed - permissions, ownership, or the file was created or renamed
b	Born	The file was created (also called ctime)

In the timeline below, the Type column shows which of the four timestamps fired at that moment. An entry reading macb means all four fired at once, which almost always means the file was newly created at that instant. An entry reading ma.. means the file already existed and was written to.

macb is the signature of something new appearing

When you scan the timeline, the macb entries are where to start. Each one is a file that did not exist a moment earlier. A file appearing on a production web server at half past two in the morning, on a host nobody was scheduled to be working on, is worth explaining.

10.2 The baseline: what normal looks like

This is the file system activity from the build of the host in May through to the day before the incident. It gives you something to compare against.

```
Date,Size,Type,Mode,UID,GID,File Name
Tue May 19 2026 03:10:14,10,ma..,rw-r--r--,0,0,"/etc/hostname"
Tue May 19 2026 03:10:14,116,ma..,rw-r--r--,0,0,"/etc/os-release"
Tue May 19 2026 03:10:20,220,ma..,rw-r--r--,0,0,"/etc/crontab"
Tue May 19 2026 03:12:44,226,macb,rw-r--r--,0,0,"/etc/apache2/apache2.conf"
Tue May 19 2026 03:12:58,190,macb,rw-r--r--,0,0,"/etc/ssh/sshd_config"
Tue May 19 2026 03:13:02,217,ma..,rw-r--r--,0,0,"/var/log/dpkg.log"
Tue May 19 2026 03:14:02,437,ma..,rw-r--r--,0,0,"/etc/apache2/sites-enabled/000-default.conf"
Tue May 19 2026 03:20:00,93,ma..,rw-r--r--,0,0,"/home/mtbadmin/.profile"
Tue Jun 02 2026 09:15:00,215,ma..,rw-r--r--,0,0,"/opt/mtb/asset-record.txt"
Tue Jun 02 2026 11:20:31,165,macb,rw-r--r--,0,0,"/var/www/html/index.html"
Tue Sep 01 2026 16:40:12,227,ma..,rw-r--r--,0,0,"/var/www/html/data/customer_export_q3.csv"
Tue Sep 01 2026 23:00:04,345,ma..,rw-r--r--,0,0,"/opt/mtb/installed-packages.txt"
```

10.3 The incident window: 2 September 2026

```
Date,Size,Type,Mode,UID,GID,File Name
Wed Sep 02 2026 02:15:22,236,macb,rw-r--r--,0,0,"/var/www/html/uploads/style.php"
Wed Sep 02 2026 02:21:05,571,ma..,rw-r--r--,0,0,"/var/log/apache2/error.log"
Wed Sep 02 2026 02:23:11,4096,ma..,drwxr-xr-x,0,0,"/home/svc-backup"
Wed Sep 02 2026 02:23:11,529,macb,rw-r--r--,0,0,"/etc/passwd"
Wed Sep 02 2026 02:23:11,89,macb,rw-r--r--,0,0,"/etc/group"
Wed Sep 02 2026 02:23:14,308,macb,rw-r--r--,0,0,"/etc/shadow"
Wed Sep 02 2026 02:24:48,194,macb,rw-----,0,0,"/root/.ssh/authorized_keys"
Wed Sep 02 2026 02:24:50,194,ma..,rw-----,0,0,"/home/svc-backup/.ssh/authorized_keys"
Wed Sep 02 2026 02:25:57,296,macb,rwxr-xr-x,0,0,"/usr/local/bin/apache-worker"
Wed Sep 02 2026 02:26:30,167,macb,rw-r--r--,0,0,"/etc/cron.d/apache-logrotate"
Wed Sep 02 2026 02:34:55,4096,ma..,drwxr-xr-x,0,0,"/tmp/.ICE-unix"
Wed Sep 02 2026 02:41:19,465,macb,rw-r--r--,0,0,"/root/.bash_history"
Wed Sep 02 2026 02:41:22,0,macb,rw-r--r--,0,0,"/home/mtbadmin/.bash_history"
Wed Sep 02 2026 09:02:55,2411,ma..,rw-r--r--,0,0,"/var/log/apache2/access.log"
Wed Sep 02 2026 09:45:01,732,ma..,rw-r--r--,0,0,"/var/log/syslog"
Wed Sep 02 2026 09:47:33,1809,ma..,rw-r--r--,0,0,"/var/log/auth.log"
```

10.4 Step by step

1. Copy the incident window timeline into a new spreadsheet tab called **Timeline**. Split it on commas so each field lands in its own column.
2. Add a column called **Interpretation**. Write one plain-English sentence for every row explaining what that entry means. Not "passwd was modified" - say what modifying it at that moment achieves for an attacker.
3. Mark every row whose Type is **macb**. These are files created during the incident. Group them: which are payload, which are persistence, which are access?
4. Two files were created 27 seconds apart at 02:25:57 and 02:26:30. Explain what those two together accomplish that neither achieves alone.
5. Look at the two entries at 02:41:19 and 02:41:22. One file has content, the other is zero bytes. Explain what the attacker was doing and why one attempt worked while the other did not.
6. Compare the incident window against the baseline in Section 10.2. Write two sentences on what makes the incident activity obviously abnormal for this host.

The size column is evidence too

One entry in the incident window has a size of 0 bytes. Empty files are not normally interesting - this one is, because of which file it is and because the equivalent file for the other account is not empty.

When you find it, ask what would have to happen for that file to end up empty, and whether the same thing happened to the equivalent file for the other account.

11. Day 3, Part 4: Log Correlation and Indicator Analysis

The timeline tells you what changed and when. The logs tell you who did it and how. Correlating the two is what turns a list of suspicious files into an account of an intrusion.

11.1 Apache access log

```
10.50.1.31 - - [01/Sep/2026:14:02:11] "GET / HTTP/1.1" 200 218 "Mozilla/5.0 (Windows NT 10.0)"
10.50.1.44 - - [01/Sep/2026:14:07:53] "GET / HTTP/1.1" 200 218 "Mozilla/5.0 (Windows NT 10.0)"
10.50.1.31 - - [01/Sep/2026:16:41:02] "GET /data/customer_export_q3.csv" 200 341 "Mozilla/5.0"
185.220.101.47 - - [02/Sep/2026:02:08:14] "GET / HTTP/1.1" 200 218 "Nmap Scripting Engine"
185.220.101.47 - - [02/Sep/2026:02:08:19] "GET /server-status" 404 452 "Nmap Scripting Engine"
185.220.101.47 - - [02/Sep/2026:02:08:24] "GET /.git/config" 404 452 "Nmap Scripting Engine"
185.220.101.47 - - [02/Sep/2026:02:09:02] "GET /icons/README" 200 1401 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:12:33] "GET /cgi-bin/.%2e/.%2e/.%2e/etc/passwd" 200 622 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:12:41] "GET /cgi-bin/.%2e/.%2e/.%2e/etc/apache2/apache2.conf" 200 289 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:13:01] "POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh" 200 89 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:13:48] "POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh" 200 142 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:15:22] "POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh" 200 61 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:17:40] "POST /uploads/style.php" 200 34 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:19:12] "POST /uploads/style.php" 200 812 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:21:05] "POST /uploads/style.php" 200 57 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:26:30] "POST /uploads/style.php" 200 19 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:34:55] "POST /uploads/style.php" 200 128 "Mozilla/5.0 (X11; Linux x86_64)"
10.50.1.31 - - [02/Sep/2026:08:14:07] "GET / HTTP/1.1" 200 218 "Mozilla/5.0 (Windows NT 10.0)"
```

Read the status codes, not just the URLs

Two requests at 02:12 use the same technique. Look at what came back: 200 with a body of 622 bytes, then 200 with 289 bytes.

A 200 means the server returned content. Compare those against the 404s two minutes earlier, and against the size of the real /etc/passwd file in the timeline. The scanner findings from Day 1 tell you what those requests were exploiting.

11.2 Authentication log

```
Sep 1 09:14:22 mtb-web01 sshd[1102]: Accepted publickey for mtbadm from 10.50.1.31 port 49221
Sep 2 02:20:47 mtb-web01 pkexec[1207]: pam_unix(polkit-1:session): session opened for user root(uid=0) by (uid=33)
Sep 2 02:21:05 mtb-web01 pkexec[1207]: www-data: Executing command [USER=root] [TTY=unknown]
[CWD=/var/www/html/uploads] [COMMAND=/bin/sh]
Sep 2 02:21:05 mtb-web01 polkitd[812]: Registered Authentication Agent for unix-process:1207
Sep 2 02:23:11 mtb-web01 useradd[1219]: new user: name=svc-backup, UID=0, GID=0,
home=/home/svc-backup, shell=/bin/bash
Sep 2 02:23:11 mtb-web01 useradd[1219]: WARNING: UID 0 is already in use by user root
Sep 2 02:23:14 mtb-web01 chpasswd[1221]: password changed for svc-backup
Sep 2 02:24:48 mtb-web01 sshd[1230]: Accepted publickey for root from 185.220.101.47 port 44118
Sep 2 02:30:01 mtb-web01 CRON[1244]: pam_unix(cron:session): session opened for user root(uid=0)
```

```
Sep  2 02:35:01 mtb-web01 CRON[1251]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 02:40:01 mtb-web01 CRON[1259]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 09:47:33 mtb-web01 sshd[1401]: Accepted publickey for mtbadm from 10.50.1.31 port 50110
```

Three lines in this log are the whole privilege story

uid=33 is the www-data account - the unprivileged identity a web server runs as. Something running as uid 33 opened a root session at 02:20:47 through pkexec.

Look at what your Day 1 assessment said was installed on this host, and which finding matched it. The privilege escalation was not a mystery; it was on your list.

The useradd warning two minutes later is the system telling you plainly that something is wrong. A new account was given the same UID as root. That is not a configuration mistake, it is a backdoor.

11.3 System log

```
Sep  2 02:23:11 mtb-web01 systemd: Created slice User Slice of UID 0.
Sep  2 02:26:31 mtb-web01 cron[812]: (*system*) RELOAD (/etc/cron.d/apache-logrotate)
Sep  2 02:30:01 mtb-web01 CRON[1244]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 02:35:01 mtb-web01 CRON[1251]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 02:40:01 mtb-web01 CRON[1259]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 08:00:01 mtb-web01 CRON[1310]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 09:45:01 mtb-web01 CRON[1396]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
```

11.4 Recovered files

Two files recovered from the image. The first is the suspicious file created at 02:15:22. The second is root's shell history, which the attacker did not clear.

/var/www/html/uploads/style.php

```
<?php
/* LAB ARTIFACT - INERT - Meridian Trust Bank forensics exercise */
$k = "a4f1";
$p = "TVRCLUxBQi1BUlRJRkFDVDogc2ltdWxhdGVkIHd1YnNoZWxsIHBheWxvYWQsIG5v
IGV4ZWNIbGFibGUgY29udGVudA==";
/* payload handler removed for lab safety */
?>
```

/root/.bash_history

```
id
uname -a
cat /etc/os-release
which curl wget python3
useradd -o -u 0 -g 0 -m -s /bin/bash svc-backup
echo 'svc-backup:REDACTED' | chpasswd
mkdir -p /root/.ssh
cat >> /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
cat > /usr/local/bin/apache-worker
chmod +x /usr/local/bin/apache-worker
cat > /etc/cron.d/apache-logrotate
systemctl restart cron
```

```
mkdir -p /tmp/.ICE-unix
tar czf /tmp/.ICE-unix/x /var/www/html/data
ls -la /tmp/.ICE-unix
history -c
```

/root/.ssh/authorized_keys

```
ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AAAAIILABPLACEHOLDERKEYNOTREALDONOTUSE00000000 mtbadmin@mtb-jump01
ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AAAAIILABPLACEHOLDERKEYNOTREALATTACKER00000000 attacker@blackfin
```

11.5 Decoding the suspicious file

The PHP file contains a base64-encoded string. Encoding is not encryption - it is a way of representing data as plain text, and it is trivially reversible. Attackers use it constantly to make payloads harder to spot in a casual read.

1. In CyberChef, clear the Recipe panel.
2. Search Operations for **From Base64** and drag **From Base64** into the Recipe. It is in the **Data format** category.
3. Paste the encoded string into the Input panel. It is printed across two lines in Section 11.4 for page width - join it into one line with no space where it wraps.

**TVRCLUxBQilBUIRJRkFDVDogc2ltdWxhdGVkIHdlYnNoZWxsIHBheWxvYWQsIG5vIGV4ZWV1dGFibG
UgY29udGVudA==**

4. Read the Output. Record what it decodes to.

MTB-LAB-ARTIFACT: simulated webshell payload, no executable content

5. Write two sentences on why an attacker would encode a payload this way, and why encoding provides no actual security.

Attackers may use Base64 encoding to make malicious content less obvious during casual inspection and to make it easier to transport as plain text. Base64 is not encryption because anyone can reverse it without a key, so it provides no real confidentiality or security.

What this file is, and what it is not

In a real incident this file would be a webshell: a small script uploaded to a web server that lets an attacker run commands through ordinary web requests. Look at the access log entries that POST to it after 02:17:40 and you can see it being used.

The file in this lab has had its working code removed and replaced with a marker string. You are learning to recognise the shape of a webshell and the traces it leaves, not handling live malicious code.

11.6 Building the consolidated IOC list

An IOC list is the deliverable another team acts on. Its whole purpose is to let someone who was not part of your investigation search their own systems for the same activity. That means it must be precise, complete, and safe to circulate.

1. Collect every indicator from the timeline, the logs, the recovered files and the acquisition record. Use CyberChef's **Extract IP addresses** on the log sections to make sure you missed nothing.
2. Sort them into types: network, file, hash, account, persistence.

3. For each indicator, record where you found it. An indicator without a source is not evidence.
4. Defang the network indicators before circulating the list. In CyberChef, search for **Defang IP** and drag **Defang IP Addresses** into the Recipe. Paste your address list in.

Why defanging matters

Defanging rewrites 185.220.101.47 as 185[.]220[.]101[.]47 so that no mail client, chat app, ticketing system or document viewer turns it into a clickable link.

IOC lists get emailed, pasted into tickets and forwarded widely. Somebody eventually clicks. Defanging is a one-second habit that prevents a colleague from connecting to attacker infrastructure by accident.

11.7 Record: consolidated IOC list

Type	Indicator (defanged)	Source	What it detects
Network	185[.]220[.]101[.]47	Alert, network captures, auth log	Suspicious external attacker IP tied to beaconing and root login
Network	185[.]220[.]101[.]47:8443	Alert / ss / lsof captures	Command-and-control connection used by apache-worker
File path	/var/www/html/uploads/style.php	Timeline, Apache log, recovered file	Uploaded webshell/payload used for attacker command activity
File path	/usr/local/bin/apache-worker	Timeline, process capture, system log	Malicious process used for recurring outbound beaconing
File hash	2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c	Acquisition record	SHA-256 identifier for style.php
File hash	1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b	Acquisition record	SHA-256 identifier for apache-worker
Account	svc-backup	Authentication log / shell history	Unauthorized UID 0 backdoor account
Persistence	/etc/cron.d/apache-logrotate	Timeline / system log / shell history	Cron persistence that executes apache-worker every five minutes
Other	attacker@blackfin SSH key	/root/.ssh/authorized_keys	Unauthorized SSH key providing persistent root access

11.8 Step by step

1. Correlate the host's local logs against the monitoring alert timeline in Section 8.1. Confirm the sequence of events lines up, and note anything that does not.

The logs line up with the monitoring alert. The attacker began reconnaissance around 02:08 UTC, exploited the web server shortly after 02:12, uploaded and used style.php, escalated privileges to root around 02:20–02:21, created persistence between 02:23 and 02:26, and then the cron job began executing apache-worker every five minutes starting at 02:30. This matches the alert showing periodic outbound connections to 185.220.101.47:8443 beginning around 02:31. No major timeline conflicts were identified.

2. Trace the suspicious process and network activity from Section 8.3 back through the logs. What was it connecting to, when did it start, and what caused it to start?

The suspicious apache-worker process connected outbound to 185.220.101.47 on port 8443. The process was created at 02:25:57, and a cron file created at 02:26:30 caused it to execute every five minutes as root. The system log confirms repeated executions beginning at 02:30:01, which explains the recurring network activity detected by monitoring.

3. Establish how the attacker moved from the web server's unprivileged account to root. Name the mechanism and cite the exact log lines.

The attacker moved from the web server account www-data to root by abusing pkexec, consistent with the PwnKit privilege-escalation finding from Day 1. The authentication log shows uid=33 opening a root session through pkexec at 02:20:47, followed by www-data executing /bin/sh as root at 02:21:05.

4. Identify every persistence mechanism the attacker installed. There are three distinct ones. Each would survive something different - a reboot, a password reset, a process being killed. Say which survives what.
 - **svc-backup UID 0 account — survives a reboot and root password reset because it is a separate root-equivalent account with its own credentials.**
 - **Attacker SSH key in /root/.ssh/authorized_keys — survives a reboot and root password change because key authentication does not depend on the root password.**
 - **Cron job /etc/cron.d/apache-logrotate — survives a reboot and restarts apache-worker if the process is killed, because cron continues launching it every five minutes.**
5. Compile your consolidated IOC list into the table in Section 11.7.

(Check above)

12. Day 3, Part 5: Root Cause and Forensic Report

You now know what happened. The remaining question is the one the bank actually cares about: why was this possible, and what stops it happening again.

12.1 Finding the root cause

Root cause is not the webshell, and it is not the beacon. Those are consequences. Root cause is the specific weakness that let an unauthenticated attacker on the internet get their first foothold on a bank's DMZ web server.

1. Look at the first request in the access log that returned a 200 with a body, from the external address, using an unusual URL structure. Note the time.

02:12:33 UTC — GET /cgi-bin/./%2e/./%2e/./%2e/./%2e/etc/passwd returned HTTP 200 with 622 bytes.

This shows the attacker successfully used path traversal to read /etc/passwd

- Match that request pattern against the findings from your Day 1 assessment. Which finding describes exactly this technique against exactly this software version?

Finding ID: MTB-0002

CVE: CVE-2021-41773

Vulnerability: Apache HTTP Server Path Traversal / RCE

Detected version: Apache 2.4.49

- Record the finding ID and the CVE. Check your Day 2 work: where did that finding rank after you applied the KEV cross-reference, and where would it have ranked on CVSS alone?

MTB-0002 (CVE-2021-41773) was the initial foothold. Although its CVSS score of 7.5 placed it outside the original top-ten findings by severity alone, threat-intelligence prioritization moved it to rank #3 because it was in CISA KEV and affected the internet-facing web server.

- Identify the second vulnerability in the chain - the one that took the attacker from the web server account to root. It is also on your Day 1 list.

After gaining web-server access, the attacker exploited MTB-0003, CVE-2021-4034 (PwnKit), to escalate from the unprivileged www-data account to root.

- Note the configuration on MTB-WEB01 that made the first vulnerability exploitable. It is printed in the evidence pack in Section 5.3. The software version alone was not sufficient.

The root cause was an exploitable Apache 2.4.49 path-traversal weakness, CVE-2021-41773, on the internet-facing MTB-WEB01 server. The Apache configuration enabled CGI through /cgi-bin/ and used <Directory /> Require all granted, allowing the attacker to traverse outside intended directories and ultimately execute commands. The attacker then exploited CVE-2021-4034 (PwnKit) to escalate from www-data to root. Both vulnerabilities had been identified during the vulnerability assessment six days earlier, demonstrating why KEV status and system exposure should be considered alongside CVSS when prioritizing remediation.

This is why Days 1 and 2 mattered

Every part of this compromise was on your assessment list six days before it happened. Nothing here was a zero-day and nothing was unknowable.

The vulnerability that gave the attacker their foothold was the one that moved up your priority list when you applied threat intelligence, and down it if you had ranked on CVSS alone.

Say that explicitly in your report. It is the strongest argument you will ever make for why vulnerability prioritisation is worth doing properly.

12.2 Building the incident timeline

Your report needs one merged timeline covering the whole incident, drawing on every source. Build it in your spreadsheet with these columns:

Time (UTC)	Event	Evidence source	Confidence
02:08:14	External attacker begins reconnaissance against MTB-WEB01 using Nmap.	Apache access log	Inferred
02:12:33	Attacker successfully exploits Apache path traversal to read /etc/passwd.	Apache access log	Inferred

Time (UTC)	Event	Evidence source	Confidence
02:15:22	style.php is created in the web uploads directory, establishing the webshell/payload.	File-system timeline + Apache access log	Confirmed
02:17:40	Attacker begins sending commands through style.php.	Apache access log + recovered style.php	Confirmed
02:20:47	www-data opens a root session through pkexec, indicating successful privilege escalation.	Authentication log + Day 1 PwnKit finding	Confirmed
02:23:11	Attacker creates svc-backup with UID 0/GID 0 as a root-equivalent backdoor account.	Authentication log + file-system timeline	Confirmed
02:24:48	Attacker establishes SSH root access using an unauthorized public key from 185.220.101.47.	Authentication log + timeline + recovered authorized_keys	Confirmed
02:25:57	Malicious /usr/local/bin/apache-worker payload is created.	File-system timeline + recovered root shell history	Confirmed
02:26:30	Cron persistence is installed to execute apache-worker every five minutes as root.	Timeline + system log + root shell history	Confirmed
02:34:55	Attacker stages web data in /tmp/.ICE-unix for possible exfiltration.	Timeline + root shell history	Confirmed
02:41:19	Attacker attempts to clear shell history, but root command history remains recoverable.	Timeline + recovered /root/.bash_history	Confirmed
09:47:12	Security monitoring alert is raised for recurring outbound connections to 185.220.101.47:8443.	SOC alert + later network/process captures	Confirmed

The Confidence column matters. Mark each entry as Confirmed if two independent sources agree, or Inferred if you are reasoning from a single source. An honest report distinguishes the two.

12.3 Prevention and monitoring recommendations

Recommendations must be specific enough that someone could implement them from your report alone. Cover all three of these:

- **Patching** - name the packages and the versions to upgrade to, and address why a host built in May was still running software with a publicly known, actively exploited flaw in September.

Upgrade Apache 2.4.49 to 2.4.51 or later and update policykit-1 to a vendor-supported version containing the PwnKit fix. The fact that this May-built host was still vulnerable in September shows the need for regular patching and faster remediation of internet-facing KEV vulnerabilities.

- **Configuration** - name the specific Apache directives that made the flaw exploitable and what they should be set to instead.

Change Apache's <Directory /> Require all granted to Require all denied and only allow access to required directories. Disable /cgi-bin/ and CGI functionality if they are not needed; otherwise restrict them tightly.

- **Detection** - at least one recommendation must be an endpoint protection or monitoring improvement that would have caught this faster. The compromise happened at 02:15 and was detected at 09:47. Explain what would have shortened that gap and by roughly how much.

Enable endpoint and file-integrity alerts for www-data launching pkexec or a shell, new UID 0 accounts, changes to /root/.ssh/authorized_keys, and new files in /etc/cron.d/. These detections could have alerted within minutes instead of waiting until 09:47, reducing the roughly 7.5-hour detection gap.

Detection is where most of the value is

The attacker was inside for seven and a half hours before anyone noticed, and they were only noticed because the beacon was noisy and predictable. A quieter one would still be running.

For each of the three persistence mechanisms you identified, ask what would have alerted on it at the moment it was created rather than hours later. Write those down as concrete detections, not as "improve monitoring".

12.4 Report structure to follow

Section	Contents
Executive summary	On September 2, MTB-WEB01 was compromised after an external attacker exploited a weakness in the internet-facing web server. The attacker gained elevated access, created multiple ways to maintain access, and staged customer web data before the activity was detected. The host was isolated, forensic evidence was preserved, and the attack path was reconstructed. Remediation should focus on patching the affected software, correcting the Apache configuration, and improving endpoint monitoring so similar activity is detected within minutes rather than hours.
Evidence handling	Volatile evidence was collected first, including running processes, network connections, logged-in users, and scheduled tasks, because this information could be lost during shutdown or imaging. The disk was then acquired at 10:31 UTC under case MTB-IR-2026-0902. SHA-256 hashes were recorded to verify integrity: style.php — 2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c; apache-worker — 1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b; /etc/cron.d/apache-logrotate — d9bc0ee387921dcaedfc87de4a72ec42e45cf268adb2777d5b014375417cb9cc; /etc/passwd — 0243860670744531f70b46cfb844a66eb8ca0de0e4b7b15dff991b3960577295. Hash verification was also reproduced in CyberChef and PowerShell.
Incident timeline	(Please look back at section 12.2)
Root cause	The initial foothold was MTB-0002, CVE-2021-41773, affecting Apache 2.4.49 on MTB-WEB01. The permissive <Directory /> Require all granted setting combined with enabled /cgi-bin/ functionality allowed the attacker to exploit path traversal and execute commands. The attacker then exploited MTB-0003, CVE-2021-4034

Section	Contents
	(PwnKit), to escalate from www-data to root. MTB-0002 had moved to rank #3 after KEV-based prioritization despite its lower CVSS score, showing why real-world exploitation and exposure should be considered alongside severity.
Indicators of compromise	185[.]220[.]101[.]47 185[.]220[.]101[.]47:8443 /var/www/html/uploads/style.php /usr/local/bin/apache-worker the SHA-256 hashes for both files • style.php SHA-256: 2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c • apache-worker SHA-256: 1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b svc-backup /etc/cron.d/apache-logrotate unauthorized attacker@blackfin SSH key
Recommendations	Patching: Upgrade Apache 2.4.49 to 2.4.51 or later and update policykit-1 to a vendor-supported version containing the PwnKit fix. Establish regular patching with expedited remediation for internet-facing KEV vulnerabilities. Configuration: Change <Directory /> Require all granted to Require all denied and explicitly allow only required directories. Disable /cgi-bin/ and CGI functionality if not needed. Detection: Alert on www-data launching shells or pkexec, new UID 0 accounts, changes to /root/.ssh/authorized_keys, and new files under /etc/cron.d/. These controls could have reduced the roughly 7.5-hour detection delay to minutes.

13. Deliverables

Submit the following through the class shared drive.

13.1 Days 1 and 2

- Your severity breakdown and top ten list.
- Your five manually validated findings, including at least one proved false positive and at least one proved true positive, with the version ranges and evidence you based each verdict on.
- Your KEV cross-reference table covering all ten CVEs.
- Your re-prioritised findings list, with a written explanation of at least one finding whose position changed.
- Your full vulnerability assessment report with remediation plan, compensating controls, false positive section, and recommended scan cadence.

13.2 Day 3

- Your evidence preservation record, including the volatile data captured, the order it was captured in, and your order-of-volatility reasoning.
- Your hash verification work from Section 9, including both hash values and what changing one character demonstrated.
- Your annotated file system timeline.
- Your log correlation notes and your consolidated, defanged IOC list.
- Your final forensic report with root cause, full merged timeline, and prevention recommendations.

14. Reflection Questions

Answer briefly in your own words, three to five sentences each. These are graded on reasoning, not length.

14.1 Days 1 and 2

1. Why can a vulnerability assessment not stop at the raw scanner output? Give a specific example from this lab of what manual validation changed.

A scanner can identify possible vulnerabilities, but it can be wrong about whether they actually apply to the installed software. In this lab, Shellshock was reported on MTB-APP01, but manual validation showed Bash 5.1.16 was outside the vulnerable range. The same process also disproved the SambaCry and BlueKeep findings. Manual validation prevented us from wasting time fixing vulnerabilities that were not actually present.

2. Give an example from this scan where CVSS score alone would have led you to the wrong priority order, and explain what threat intelligence corrected.

CVE-2021-41773 had a CVSS score of 7.5, while the vsftpd vulnerability had a score of 9.8. CVSS alone would have put the 9.8 issue first, but CVE-2021-41773 was in KEV and affected the internet-facing web server. Threat intelligence showed that the lower-scored vulnerability had confirmed real-world exploitation and greater exposure, so it deserved higher priority.

3. All three false positives in this scan were in the KEV catalog. Explain why that combination is more dangerous to a security team than a false positive that is not in KEV.

A KEV listing makes a vulnerability look especially urgent because it is known to be exploited in the real world. If a scanner falsely reports one of those vulnerabilities, a security team may immediately divert resources toward fixing something that does not actually affect the host. In this lab, Shellshock, SambaCry, and BlueKeep were all in KEV but were false positives on the systems scanned. This shows why KEV status should increase urgency only after the finding is validated.

4. Why is a compensating control still valuable even when it does not fix the underlying vulnerability?

A compensating control reduces the opportunity for an attacker even when the underlying vulnerability cannot be fixed immediately. For example, restricting legacy FTP to approved internal systems would reduce exposure while the service is still required. It also gives the security team time to plan a permanent fix without simply leaving the weakness completely open. The vulnerability remains, but the risk is lowered and monitored.

14.2 Day 3

1. Why does the order you collect evidence in matter, and what specifically would have been lost if the disk had been imaged before the running processes and network connections were captured?

Some evidence disappears or changes much faster than disk data, so volatile information needs to be captured first. If we had focused on disk imaging before checking the live system, the apache-worker process or its connection to 185.220.101.47:8443 could have ended before we documented them. We also could have lost information about active sessions, logged-in users, and other running processes. Capturing those first preserved evidence that might not have existed later.

2. How did correlating the host logs with the monitoring alert change or confirm your understanding of what happened?

The monitoring alert told us that suspicious outbound beaconing was happening, but the host logs explained how the attacker reached that point. The Apache log showed reconnaissance and exploitation, the authentication log showed www-data escalating to root, and the system log showed the cron job repeatedly launching apache-worker. Together, they confirmed that the beacon was part of a larger compromise rather than an isolated suspicious connection.

3. Why is identifying the root cause more valuable to the bank than simply removing the malicious process and the backdoor account?

Removing apache-worker or deleting the backdoor account would stop some of the attacker's activity, but it would not fix the weakness that allowed the compromise. The root cause showed that vulnerable Apache software, a permissive configuration, and PwnKit created the attack path. Fixing those problems prevents the same method from being used again. Root-cause analysis therefore helps the bank prevent recurrence instead of only cleaning up the symptoms.

4. The attacker cleared one shell history file and missed another. What does that tell you about relying on any single evidence source, and how did the timeline let you detect the deletion even though the contents were gone?

The attacker successfully left one shell-history file at zero bytes, but the root history still contained commands that reconstructed much of the attack. This shows why investigators should never depend on a single evidence source, because attackers may delete or alter it. The filesystem timeline still showed that the empty history file was created or changed at a suspicious time, even though its contents were gone. Combining metadata, logs, and recovered files allowed us to identify the anti-forensics attempt anyway.

15. Grading Alignment

This lab counts toward your lab-completion and documentation-quality grading criteria.

16. Completion Checklist

Work down this list before you submit. Every line is something that is graded. If you cannot tick a line, that is where to spend your remaining time.

16.1 Day 1

Done	Item
	All 37 findings copied into your spreadsheet
	Severity counts recorded, and the five counts add up to 37
	Top ten by CVSS recorded, with the host for each
	At least one finding flagged as suspicious, with a reason written down
	Five findings picked for validation, covering a mix of severities
	Each of the five looked up on NVD, with the affected range recorded
	At least one false positive proved, quoting the range and the detected version
	At least one true positive proved, with the exact confirming evidence
	Any severity changes noted, with the reason for each

16.2 Day 2

Done	Item
	All ten CVEs checked against KEV, including the ones you disproved
	KEV date added recorded for every CVE that is listed
	Findings re-prioritised so confirmed exploitation outweighs CVSS alone
	False positives removed from the priority list entirely
	At least one position change explained in writing
	Publication date and KEV date recorded for your top finding, with the gap noted
	Report written with all six sections from Section 7.3
	Every finding has a specific fix, not "patch it"
	Compensating control proposed for anything that cannot be patched now
	Scan cadence recommended and justified
	Executive summary written with no CVE numbers and no CVSS scores

16.3 Day 3

Done	Item
	Alert trigger recorded and quoted
	All five volatile captures reviewed and the host state written up
	Initial IOC table filled in, all six rows
	The out-of-place login in Capture 3 identified and explained
	Order-of-volatility reasoning written, referencing the volatility table
	Both hash values from Section 9 reproduced and matched
	Tampering demonstration completed and written up
	Timeline copied into a spreadsheet and split into columns
	Every timeline row given a plain-English interpretation
	All macb rows identified and grouped as payload, persistence or access
	The two files created 27 seconds apart explained together
	The zero-byte file explained
	Privilege escalation mechanism named, with the log lines cited
	All three persistence mechanisms identified, with what each survives
	Base64 payload decoded and recorded
	Consolidated IOC list completed and defanged
	Root cause identified, with the CVE and your Day 1 finding ID
	Second vulnerability in the chain identified
	The enabling configuration named
	Merged incident timeline built, with confidence marked on every row
	Recommendations cover patching, configuration and detection
	Executive summary written for a non-technical reader

Before you submit

Read your two executive summaries out loud. If you stumble, or if you used a word a branch manager would not know, rewrite them.

Check that every factual claim about a CVE in your reports came from NVD or the CISA catalog and not from memory or from an AI assistant.

Check that no live, clickable attacker address survives anywhere in your IOC list.

Appendix A: Reference Data and Troubleshooting

Everything in this appendix is here so that nothing can stop you. If a website is slow, blocked or down, the values you would have looked up are printed below. Use them, and note in your report that you used the printed reference rather than the live source.

A.1 CVE affected version ranges

These are the affected ranges as published by the vendors and recorded on NVD. Compare each against the detected version in Section 3.2.

CVE	Affected versions	Detected on host	Verdict
CVE-2021-42013	Apache httpd 2.4.49 and 2.4.50 only	Apache 2.4.49	In range
CVE-2021-41773	Apache httpd 2.4.49 only	Apache 2.4.49	In range
CVE-2021-4034	polkit pkexec, all versions from 2009 until the January 2022 fix	policykit-1 0.105-31	In range
CVE-2020-11022	jQuery 1.2 up to but not including 3.5.0	jQuery 3.4.1	In range
CVE-2011-2523	vsftpd 2.3.4 only (backdoored release tarball)	vsftpd 2.3.4	In range
CVE-2014-6271	GNU bash through 4.3	GNU bash 5.1.16	OUT of range
CVE-2017-7494	Samba 3.5.0 up to but not including 4.6.4	Samba 4.15.13	OUT of range
CVE-2023-4911	glibc 2.34 through 2.37 in affected builds	glibc 2.35	In range
CVE-2019-0708	Windows XP, Server 2003, Vista, 7, Server 2008, Server 2008 R2	Windows Server 2019	OUT of range
CVE-2020-1472	Windows Server 2008 R2 through 2019, unpatched since Aug 2020	Windows Server 2019, last update June 2020	In range

Do the NVD lookups yourself first. This table is a safety net and a way to check your work, not a shortcut past the exercise. Your report must show the reasoning, not just the verdict.

A.2 KEV catalog status

Status as recorded in the CISA Known Exploited Vulnerabilities catalog. Verify each one live on the CISA site during Section 5.

CVE	CVSS	KEV status	Date added to KEV
CVE-2021-42013	9.8	In KEV	2021-11-03
CVE-2021-41773	7.5	In KEV	2021-11-03
CVE-2021-4034	7.8	In KEV	2022-06-27
CVE-2020-11022	6.1	Not in KEV	-
CVE-2011-2523	9.8	Not in KEV	-
CVE-2014-6271	9.8	In KEV	2022-01-28

CVE	CVSS	KEV status	Date added to KEV
CVE-2017-7494	9.8	In KEV	2023-03-30
CVE-2023-4911	6.5	In KEV	2023-11-21
CVE-2019-0708	9.8	In KEV	2021-11-03
CVE-2020-1472	8.8	In KEV	2021-11-03

A.3 CyberChef quick reference

Every operation used in this lab, where to find it, and what it does. Type the operation name into the search box at the top left of the Operations panel.

Operation	Category	Used in	What it does
Extract IP addresses	Extractors	7.4, 10.6	Pulls every IPv4 and IPv6 address out of a block of text
Sort	Utils	7.4	Sorts the output lines
SHA2	Hashing	8.3, 8.4	Calculates a SHA-256 fingerprint of the input
From Base64	Data format	10.5	Decodes a base64 string back to readable text
Defang IP Addresses	Networking	10.6	Rewrites addresses so they cannot become clickable links

Three CyberChef habits worth learning now

Operations stack. Everything in the Recipe panel runs top to bottom, so you can chain Extract IP addresses into Sort into Defang IP Addresses and get a finished IOC list in one pass.

The Output updates as you type. There is no run button to press and nothing to submit.

To start over, click the bin icon at the top of the Recipe panel. Leftover operations from a previous exercise are the usual reason output looks wrong.

A.4 If something is not working

Problem	What to do
My SHA-256 output does not match the printed value	You have an extra space or line break in the Input. Clear it with Ctrl+A then Delete and type the line by hand. The character count at the bottom of the Input panel should read 34.
CyberChef output looks wrong or empty	An operation from a previous exercise is still in the Recipe panel. Click the bin icon at the top of the Recipe panel to clear it, then add only the operation you need.
CyberChef will not load at all	Use the built-in operating system commands in Section 8.5 for hashing. For decoding base64 and extracting addresses, the printed evidence is short enough to read by hand. Note the substitution in your report.
The NVD page is slow or will not load	Use the printed ranges in Appendix A.1 and say so in your report. Retry NVD later if you want to confirm.
I cannot find a CVE in the KEV search box	Check for a typo first, then confirm against Appendix A.2. Not finding a CVE is often the correct answer - one of the nine is genuinely not in the catalog.

Problem	What to do
I do not know which findings to validate	Start with the three Critical 9.8 findings that have CVE numbers. Section 4.5 points you at each of them without giving the answer away.
I am stuck on Day 3 and cannot see the chain	Work backwards. Start at the alert in Section 7.1, find the process in Section 7.3, find where that process came from in Section 9.3, then find who created it in Section 10.2.

A.5 Working with an AI assistant

You may use an AI assistant on this lab, and you should. Use it the way a working analyst does.

- **Good use** - ask it to explain what a pkexec log line means, what a base64 string is, why a UID 0 warning matters, or to review whether your executive summary is clear to a non-technical reader.
- **Bad use** - asking it which findings are false positives. You will get an answer that sounds confident and you will not be able to defend it, because you did not do the version comparison yourself.
- **Always verify** - AI assistants are frequently wrong about specific CVE version ranges and KEV membership. Every factual claim about a CVE must be confirmed against NVD or the CISA catalog before it goes in your report. That verification habit is itself part of what Week 7 covers.

Cite it. If an AI helped you reach a conclusion, note where and what you did to verify it. That is the standard a regulated bank holds analysts to, and it is the standard here.